

PARTIU MARROCOS / KEROMINDPARTIU MARROCOS / KEROMIND

Booking Foundation 01 — Relatório de Fechamento

Relatório de Fechamento

15 de setembro de 2026

Etapas 1 de 8 — PM-NIGHT-RUN-02 (COMANDO DE EXECUÇÃO CONTINUADA CONTROLADA)

Executada sob autorização recém-concedida ("AUTORIZADA"), logo após o fechamento de PM-NIGHT-RUN-01.

Objetivo

Transformar uma Proposal ACEITA numa reserva/operação comercial real (Booking), com máquina de estados explícita e passageiros associados — primeiro elo do fluxo LEAD → PROPOSTA → ACEITE → **BOOKING** → pagamento → financeiro → documentação → viagem.

Arquitetura

Booking não duplica dados da Proposal

Decisão explícita (o próprio comando pede "não duplicar dados da Proposal sem necessidade" e "quando precisa ser congelado, usar snapshot"): uma Proposal **ACEITA** já É o snapshot imutável (nunca é reescrita depois de **ENVIADA**, ver Proposal Foundation 01). **Booking** só guarda a FK (**proposalId**) — roteiro/preço/moeda/datas/passageiros/condições continuam sendo lidos via **booking.proposal.***, nunca duplicados. Isso elimina de raiz qualquer risco de divergência entre "o que foi vendido" e "o que está registrado na reserva".

Idempotência estrutural, não de aplicação

@unique([tenantId, proposalId]) no schema: uma Proposal só pode gerar UM Booking — o próprio banco rejeita uma segunda tentativa, não é uma checagem que a aplicação poderia esquecer de fazer. **criarBookingDaProposta** trata isso de forma amigável (retorna o Booking já existente com **criado: false** em vez de lançar erro) — testado explicitamente contra duplo-clique/retry.

Máquina de estados explícita

BookingStatus: AGUARDANDO_PAGAMENTO → PAGAMENTO_PARCIAL/PAGO → AGUARDANDO_DOCUMENTOS/CONFIRMADA → EM_OPERACAO → CONCLUIDA, com **CANCELADA** alcançável de qualquer estado não-terminal (exceto **EM_OPERACAO** — cancelar uma viagem já em andamento é decisão operacional excepcional, fora desta fundação). Tabela pura **TRANSICOES_VALIDAS** (mesmo padrão de **transicaoValida** em Gates/T1) valida toda transição antes do UPDATE — testada isoladamente (7 testes unitários) e contra o banco de verdade (transição inválida não muda nada, estado terminal nunca sai do lugar).

Os estados **PAGAMENTO_PARCIAL/PAGO** já existem no enum desde já (mesmo sem Payment Foundation construído ainda) — evita uma migration extra na Etapa 2 só para adicionar valores de enum; quem vai efetivamente *disparar* essas transições a partir de pagamentos reais é a Etapa 2.

Travelers (passageiros) — consumidor real já autorizado nesta mesma janela

Diferente da decisão de "manter como contrato" tomada em Finance Core (PM-NIGHT-RUN-01), aqui HÁ um consumidor real e próximo: a Etapa 4 desta mesma autorização (Travel Document Foundation) referencia **Traveler** para requisito/documento por pessoa. Por isso **Traveler** foi promovido a tabela real agora, não deixado como contrato.

Privacy-by-design (§8/§29 do comando): nenhum número de passaporte, nenhum documento sensível, nenhuma imagem — só nome/tipo/data de nascimento/nacionalidade/contato opcional. Status documental e upload de arquivo são explicitamente Etapa 4, com arquitetura de storage segura própria, não inventada aqui.

Migrations

2 novas, ambas puramente aditivas (2 tabelas novas, nenhuma alteração em tabela existente):

```
20260915040000_booking_foundation_01          -- enums BookingStatus/TravelerTipo + tabelas bookings/travelers
20260915040100_enable_rls_booking_foundation_01 -- RLS (mesmo padrão tenant_isolation de sempre)
```

33 migrations no total (era 31 ao final de PM-NIGHT-RUN-01).

RBAC

2 permissões novas: **bookings.view** (ver reservas/passageiros) e **bookings.manage** (criar reserva a partir de proposta aceita, alterar status, gerenciar passageiros). Administrador ganha ambas automaticamente; Vendas ganha ambas; Atendimento ganha só **bookings.view**.

Segurança

- RLS (**FORCE ROW LEVEL SECURITY** + policy **tenant_isolation**) em ambas as tabelas desde a migration que as cria — testado ativamente (Tenant B não lista, não cria via Proposal de outro tenant — nem consegue ENXERGAR a Proposal por RLS, então nunca chega a criar Booking indevido —, e não lê Booking do Tenant A por id direto).
- FK composta (**tenantId, proposalId**)/(**tenantId, leadId**)/(**tenantId, bookingId**) em toda referência entre tabelas tenant-scoped — mesmo padrão estabelecido desde o Tenant Core.
- **BOOKING_CRIADO** e **BOOKING_STATUS_ALTERADO** auditados (T1), com **de/para** no detalhe de toda transição.

Testes novos (19)

- **packages/db/tests/unit/booking-transicoes.test.ts** — 7 (máquina de estados pura: caminho feliz completo, cancelamento em cada estado não-terminal, estados terminais sem saída, sem retrocesso, EM_OPERACAO não cancela, sem autotransição).
- **packages/db/tests/integration/booking.test.ts** — 12 (criação a partir de ACEITA, rejeição de RASCUNHO/ENVIADA, idempotência estrutural real contra o banco, duas Propostas geram dois Bookings, transição válida auditada, transição inválida rejeitada sem mudar nada, terminal não sai do lugar, travelers CRUD, isolamento multi-tenant em 3 cenários).

Regressão

378/378 testes passando (359 ao final de PM-NIGHT-RUN-01 + 19 novos). Typecheck limpo (**packages/db** e **apps/web**). Build limpo — 17 rotas (**/leads/[id]** cresceu de 3.49kB pra 4.38kB — painel de reserva).

Verificação ponta a ponta (navegador real)

Fluxo completo exercido de verdade contra o app rodando: lead existente → nova proposta (R\$ 9.000, sem preço de referência, sem gatilho de política) → enviada direto (sem Gate) → aceita → **"Criar reserva"** aparece e cria o Booking (**AGUARDANDO_PAGAMENTO**) → passageiro "Maria Silva" adicionado → botão "Pago" clicado → status muda pra **PAGO** de verdade e os botões de próxima transição atualizam corretamente (**AGUARDANDO_DOCUMENTOS/CONFIRMADA/CANCELADA**) → botão "Criar reserva" desaparece (booking já existe, confirma idempotência também na UI). Zero erros de console em todo o fluxo. Dados de verificação (proposta/booking/passageiro de teste, senha temporária do admin) removidos/restaurados ao final.

Limitações

PAGAMENTO_PARCIAL/PAGO hoje só são alcançáveis manualmente (sem Payment real ainda — isso é a Etapa 2); **Traveler** não tem status documental (Etapa 4); nenhum vínculo com Trip/Operação ainda (Etapa 5); **market/moeda/valor** não são campos próprios do Booking — sempre lidos via **booking.proposal** (decisão deliberada, não uma lacuna).

Decisões do fundador pendentes

Nenhuma nova.

Resultado / Status

GREEN — implementação completa dentro do escopo, 19/19 testes novos passando, regressão completa (378/378), typecheck/build limpos, 2 migrations aditivas com RLS desde o primeiro commit, idempotência estrutural (não só de aplicação), máquina de estados testada isoladamente e contra o banco, RBAC estendido corretamente, nenhuma perda de dados, nenhuma decisão humana crítica pendente, verificação ponta a ponta completa em navegador real.

Próximo bloco

Etapa 2 — Payment Foundation. **Prosseguindo automaticamente conforme autorização (PM-NIGHT-RUN-02, "não é necessário solicitar autorização entre etapas GREEN")**.